Python Programming (ABL Assignment Report)

Name: [Your Name] | **Roll No:** [Your Roll No]

CASE STUDY 1: ATTENDANCE RECOVERY CALCULATOR

1. Problem Statement & Objectives

Calculate the minimum number of additional consecutive classes required to reach a target attendance percentage.

2. Assumptions & Scope

- 100% attendance in future classes.
- Total conducted classes > 0 and target percentage ≤ 100%.

3. Algorithm

1. Input conducted, attended, and target percentage.
2. Validate inputs; loop while current percentage < target.
3. Increment conducted, attended, and extra classes counter.
4. Print final percentage and required extra classes.

4. Pseudocode

```
      WHILE (attended / conducted) * 100 < target: conducted = conducted + 1 attended = attended + 1
extra = extra + 1 END WHILE
```

5. Flowchart

```
[Start] → [Input Details] → <Current % < Target?> —Yes→ [Increment Counts] → Loop
↓ No
[Print Required Classes] → [End]
```

6. Python Source Code (case01_attendance_recovery.py)

```
      conducted = int(input("Enter total classes conducted: ")) attended = int(input("Enter classes
attended: ")) target = float(input("Enter required attendance percentage: ")) if conducted <= 0 or a
ttended < 0 or attended > conducted: print("Invalid class details.") elif target < 0 or target > 10
0: print("Target must be between 0 and 100.") else: current = attended / conducted * 100 extra = 0 w
hile current < target: conducted += 1 attended += 1 extra += 1 current = attended / conducted * 100
print("Current attendance:", round(attended / conducted * 100, 2), "%") if extra == 0: print("You ha
ve already reached the target.") else: print("Additional consecutive classes required:", extra)
```

7. Test Cases & Sample Output

Inputs	Execution Output
50, 30, 75	Current attendance: 75.0 % | Additional consecutive classes required: 30
40, 35, 75	Current attendance: 87.5 % | You have already reached the target.

8. Conclusion

The program calculates required extra classes correctly using a while loop and handles invalid user inputs properly.

CASE STUDY 2: ATM TRANSACTION SIMULATOR

1. Problem Statement & Objectives

Simulate an ATM machine that processes deposits and withdrawals while tracking valid and rejected transactions.

2. Assumptions & Scope

- Initial balance and transaction count ≥ 0.
- Supported transaction types: 'D' (Deposit), 'W' (Withdrawal).

3. Algorithm

1. Input initial balance and number of transactions N.
2. Loop N times, taking transaction type and amount.
3. Validate transaction and update balance or increment rejected counter.
4. Display summary of balance, deposits, withdrawals, and rejected count.

4. Pseudocode

```
      FOR i = 1 to N: IF type == 'D' AND amt > 0 THEN balance += amt; deposits++ ELSE IF type == 'W'
AND amt <= balance THEN balance -= amt; withdrawals++ ELSE rejected++ END FOR
```

5. Flowchart

```
[Start] → [Input Balance & N] → [Loop N Times] → <Valid Transaction?> —Yes→ [Update Balance]
↓ No
[Increment Rejected Count] → [Print Transaction Summary] → [End]
```

6. Python Source Code (case02_atm_simulation.py)

```
      balance = float(input("Enter initial balance: ")) n = int(input("How many transactions? ")) de
posits = 0 withdrawals = 0 rejected = 0 if balance < 0 or n < 0: print("Invalid starting balance or
transaction count.") else: for i in range(n): kind = input("Enter D for deposit or W for withdrawal:
").upper() amount = float(input("Enter amount: ")) if amount <= 0: print("Invalid amount. Transactio
n rejected.") rejected += 1 elif kind == "D": balance += amount deposits += 1 print("Deposit success
ful.") elif kind == "W": if amount <= balance: balance -= amount withdrawals += 1 print("Withdrawal
successful.") else: print("Insufficient balance. Transaction rejected.") rejected += 1 else: print
("Invalid transaction type.") rejected += 1 print("\nTransaction Summary") print("Final balance:", r
ound(balance, 2)) print("Successful deposits:", deposits) print("Successful withdrawals:", withdrawa
ls) print("Rejected transactions:", rejected)
```

7. Test Cases & Sample Output

Inputs	Execution Output
Bal: 1000, N=1, D 500	Deposit successful. | Final balance: 1500.0 | Successful deposits: 1
Bal: 500, N=1, W 800	Insufficient balance. | Final balance: 500.0 | Rejected transactions: 1

8. Conclusion

Simulates real-world ATM logic accurately, enforcing balance rules and logging rejected transactions.

CASE STUDY 3: ACADEMIC PERFORMANCE TREND ANALYZER

1. Problem Statement & Objectives

Compare student performance across two exams and classify the trend as Improved, Declined, or No Change.

2. Assumptions & Scope

- Subjects: Python, Maths, English.
- Marks must be strictly between 0 and 100 for each exam.

3. Algorithm

1. Loop through subject list, prompting Exam 1 and Exam 2 marks.
2. Validate mark ranges; terminate if invalid.
3. Compare exam scores per subject and determine performance status.
4. Output subject-wise status and total mark difference.

4. Pseudocode

```
FOR subject IN subjects: IF mark1 < 0 OR mark2 > 100 THEN BREAK exam1.append(mark1); exam2.app
end(mark2) ELSE: COMPUTE score differences and total overall change
```

5. Flowchart

```
[Start] → [Input Marks for Subjects] → <Valid Marks (0-100)?> —No→ [Print Error & Break]
↓ Yes
[Compare Exam 2 vs Exam 1] → [Print Subject Trends & Totals] → [End]
```

6. Python Source Code (case03_performance_trend.py)

```
subjects = ["Python", "Maths", "English"] exam1 = [] exam2 = [] for subject in subjects: first
= float(input("Enter " + subject + " marks in Exam 1: ")) second = float(input("Enter " + subject +
" marks in Exam 2: ")) if first < 0 or first > 100 or second < 0 or second > 100: print("Marks must
be between 0 and 100. Please restart and enter valid marks.") break exam1.append(first) exam2.append
(second) else: print("\nPerformance Trend") for i in range(len(subjects)): if exam2[i] > exam1[i]: s
tatus = "Improved" elif exam2[i] < exam1[i]: status = "Declined" else: status = "No change" print(su
bjects[i], ":", status, "(change:", exam2[i] - exam1[i], ")") print("Exam 1 total:", sum(exam1)) pri
nt("Exam 2 total:", sum(exam2)) print("Overall change:", sum(exam2) - sum(exam1))
```

7. Test Cases & Sample Output

Inputs (Exam 1, Exam 2)	Execution Output
[70,80,85], [80,75,85]	Python: Improved (+10), Maths: Declined (-5), English: No change (0)
[90, 105, 80]	Marks must be between 0 and 100. Please restart and enter valid marks.

8. Conclusion

Demonstrates parallel list processing and Python's for-else construct for clean data reporting.

CASE STUDY 4: PASSWORD VALIDATION SYSTEM

1. Problem Statement & Objectives

Validate a password string against complexity rules and list all specific rule violations.

2. Assumptions & Scope

- Min 8 chars, 1 uppercase, 1 lowercase, 1 digit, 1 special char.
- Special characters are verified using non-alphanumeric check.

3. Algorithm

1. Read input password string.
2. Evaluate length, uppercase, lowercase, digit, and special characters.
3. Store rule failure messages in an error list.
4. If error list is empty, output success; otherwise list all errors.

4. Pseudocode

```
IF len(pwd) < 8 THEN errors.append("Use at least 8 characters.") IF NOT has_upper THEN errors.append("Add at least one uppercase letter.") IF NOT has_special THEN errors.append("Add at least one special character.") IF len(errors) == 0 THEN Print "Success" ELSE Print errors
```

5. Flowchart

```
[Start] → [Input Password] → [Evaluate 5 Security Rules] → <Errors Empty?> —Yes→ [Print Success]
↓ No
[Display List of Errors] → [End]
```

6. Python Source Code (case04_password_validation.py)

```
password = input("Enter a sample password to check: ") errors = [] if len(password) < 8: errors.append("Use at least 8 characters.") if not any(ch.isupper() for ch in password): errors.append("Add at least one uppercase letter.") if not any(ch.islower() for ch in password): errors.append("Add at least one lowercase letter.") if not any(ch.isdigit() for ch in password): errors.append("Add at least one digit.") if not any(not ch.isalnum() for ch in password): errors.append("Add at least one special character.") if len(errors) == 0: print("Password meets all sample rules.") else: print("Password does not meet the sample rules:") for error in errors: print("-", error)
```

7. Test Cases & Sample Output

Input Password	Execution Output
`Pass@1234`	Password meets all sample rules.
`hello`	Errors: Use at least 8 characters, Add uppercase, Add digit, Add special char.

8. Conclusion

Uses string methods and generators effectively to validate user input against security policies.

CASE STUDY 5: MCQ ANSWER EVALUATOR

1. Problem Statement & Objectives

Evaluate student MCQ responses against an answer key and compute total score with negative marking.

2. Assumptions & Scope

- Answer key: ['A', 'C', 'B', 'D', 'A'].
- Blank response is '-'. Scoring: Correct +1, Incorrect -0.25, Blank 0.

3. Algorithm

1. Input student answers as space-separated values.
2. Verify answer count matches key length.
3. Loop through answers: count correct, incorrect, and unanswered.
4. Calculate final score = correct - (incorrect * 0.25).

4. Pseudocode

```
FOR i = 0 to 4: IF ans[i] == '-' THEN unanswered++ ELSE IF ans[i] == key[i] THEN correct++ ELS
E incorrect++ score = correct - (incorrect * 0.25)
```

5. Flowchart

```
[Start] → [Input Answers] → [Loop & Compare with Key] → [Compute Score: Correct - 0.25*Incorrect]
↓
[Print Score & Summary] → [End]
```

6. Python Source Code (case05_answer_evaluation.py)

```
answer_key = ["A", "C", "B", "D", "A"] answers = input("Enter 5 answers separated by spaces (u
se - for blank): ").upper().split() if len(answers) != len(answer_key): print("Please enter exactl
y", len(answer_key), "answers.") else: correct = 0 incorrect = 0 unanswered = 0 for i in range(len(a
nswer_key)): if answers[i] == "-": unanswered += 1 elif answers[i] == answer_key[i]: correct += 1 el
se: incorrect += 1 score = correct - incorrect * 0.25 print("Correct answers:", correct) print("Inco
rrect answers:", incorrect) print("Unanswered questions:", unanswered) print("Final score:", round(s
core, 2), "out of", len(answer_key))
```

7. Test Cases & Sample Output

Input Answers	Execution Output
`A C B D A`	Correct: 5 | Incorrect: 0 | Unanswered: 0 | Final score: 5.0 out of 5
`A B B - A`	Correct: 3 | Incorrect: 1 | Unanswered: 1 | Final score: 2.75 out of 5

8. Conclusion

Accurately evaluates student performance using list comparisons and floating-point arithmetic.

CASE STUDY 6: MARKS MATRIX ANALYSIS

1. Problem Statement & Objectives

Process a 2D matrix of student marks to calculate individual totals, subject averages, and top performer.

2. Assumptions & Scope

- 3 Students (Asha, Ravi, Sara), 3 Subjects (Python, Maths, English).
- Data organized in 2D matrix list `marks[student][subject]`.

3. Algorithm

1. Sum rows for individual student totals and averages.
2. Iterate columns to calculate overall subject averages.
3. Identify index of maximum total score to find top performer.
4. Display all computed metrics clearly.

4. Pseudocode

```
      FOR row IN marks: totals.append(sum(row)) FOR col j = 0 to 2: column_total = sum(marks[i][j] f
or i in 0..2) top_performer = students[totals.index(max(totals))]
```

5. Flowchart

```
[Start] → [Initialize Marks Matrix] → [Calculate Student Totals & Averages]
↓
[Calculate Subject Averages] → [Locate Highest Marks & Print Top Performer] → [End]
```

6. Python Source Code (case06_marks_matrix.py)

```
      students = ["Asha", "Ravi", "Sara"] subjects = ["Python", "Maths", "English"] marks = [ [85, 7
8, 90], [72, 88, 80], [92, 91, 89] ] totals = [] for i in range(len(marks)): total = sum(marks[i]) t
otals.append(total) print(students[i], "total:", total, "average:", round(total / len(subjects), 2))
print("\nSubject averages:") for j in range(len(subjects)): column_total = 0 for i in range(len(mark
s)): column_total += marks[i][j] print(subjects[j], ":", round(column_total / len(students), 2)) top
_index = totals.index(max(totals)) print("Top performer:", students[top_index], "with", totals[top_i
ndex], "marks")
```

7. Test Cases & Sample Output

Metric Evaluated	Execution Output
Student Totals	Asha: 253 (Avg: 84.33) | Ravi: 240 (Avg: 80.0) | Sara: 272 (Avg: 90.67)
Subject Averages & Top	Python: 83.0 | Maths: 85.67 | English: 86.33 | Top: Sara (272)

8. Conclusion

Demonstrates 2D list manipulation and row/column nested loop iterations.

CASE STUDY 7: PRODUCT CATALOGUE INVENTORY AUDIT

1. Problem Statement & Objectives

Audit product tuples in a catalogue list, compute inventory valuations, and trigger restocking alerts.

2. Assumptions & Scope

- Tuple structure: `(product_id, price, quantity)`.
- Low stock replenishment alert triggered if quantity < 5.

3. Algorithm

1. Loop through product tuples in catalogue list.
2. Unpack tuple into `product_id, price, quantity`.
3. Validate price and quantity values.
4. Calculate inventory value; alert if stock < 5; compute total.

4. Pseudocode

```
      FOR (id, price, qty) IN products: IF price < 0 OR qty < 0 THEN Print "Invalid record" ELSE val
= price * qty; total_value += val IF qty < 5 THEN Print "Replenishment needed"
```

5. Flowchart

```
[Start] → [Iterate Product Tuples] → [Unpack & Validate Record] → [Compute Inventory Value]
↓
<Quantity < 5?> —Yes→ [Print Low Stock Alert] → [Output Total Inventory Value] → [End]
```

6. Python Source Code (case07_product_catalogue.py)

```
      products = [ ("P101", 50.0, 10), ("P102", 120.0, 3), ("P103", 75.0, 0), ("P104", 40.0, 8) ] to
tal_value = 0 print("Product Catalogue Audit") for product in products: product_id, price, quantity
= product if price < 0 or quantity < 0: print(product_id, "- Invalid record") else: value = price *
quantity total_value += value print(product_id, "inventory value:", value) if quantity < 5: print("
Replenishment needed; current stock:", quantity) print("Total inventory value:", round(total_value,
2))
```

7. Test Cases & Sample Output

Product Sample	Execution Output
P102 (120.0, 3)	P102 inventory value: 360.0 | Replenishment needed; current stock: 3
Overall Inventory	Total inventory value: 1180.0

8. Conclusion

Uses tuple unpacking efficiently to process structured records and perform data audits.

CASE STUDY 8: CANTEEN FOOD WASTAGE MONITOR

1. Problem Statement & Objectives

Monitor daily meal preparation and consumption to calculate food wastage percentage and highlight excess waste.

2. Assumptions & Scope

- 5 days data (Mon-Fri).
- Served meals ≤ prepared meals. Warning threshold = 10%.

3. Algorithm

1. Loop through day indices 0 to 4.
2. Compute leftovers = prepared - served.
3. Calculate wastage percentage = (leftovers / prepared) * 100.
4. Output results and trigger warning if wastage % > threshold.

4. Pseudocode

```
FOR i = 0 to len(days)-1: leftovers = prepared_meals[i] - served_meals[i] waste_percent = (lef
tovers / prepared_meals[i]) * 100 IF waste_percent > 10 THEN Print "Warning: Wastage exceeded"
```

5. Flowchart

```
[Start] → [Loop Weekdays] → [Leftovers = Prepared - Served] → [Calculate Wastage %]
↓
<Wastage % > 10%?> —Yes→ [Print Warning Alert] → [End]
```

6. Python Source Code (case08_food_waste.py)

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"] prepared_meals = [100, 120, 9
0, 110, 100] served_meals = [92, 100, 85, 90, 70] threshold = 10 for i in range(len(days)): prepared
= prepared_meals[i] served = served_meals[i] if prepared <= 0 or served < 0 or served > prepared: pr
int(days[i], "- Invalid meal counts") else: leftovers = prepared - served waste_percent = leftovers
/ prepared * 100 print(days[i], "leftovers:", leftovers, "wastage:", round(waste_percent, 2), "%") i
f waste_percent > threshold: print(" Warning: Wastage exceeded", threshold, "%")
```

7. Test Cases & Sample Output

Day (Prepared/Served)	Execution Output
Tuesday (120 / 100)	Tuesday leftovers: 20 wastage: 16.67 % | Warning: Wastage exceeded 10 %
Friday (100 / 70)	Friday leftovers: 30 wastage: 30.0 % | Warning: Wastage exceeded 10 %

8. Conclusion

Tracks operational metrics using parallel list indexing and raises conditional alerts.

CASE STUDY 9: CANTEEN BILLING & STOCK MANAGER

1. Problem Statement & Objectives

Process item orders, update inventory stock dynamically, and calculate total daily sales.

2. Assumptions & Scope

- Items: Sandwich (Rs. 50), Samosa (Rs. 15), Juice (Rs. 25).
- Orders exceeding available stock are rejected.

3. Algorithm

1. Display menu with item prices and current stock.
2. Prompt total orders count; loop to get item number and quantity.
3. Validate choice and stock level; deduct stock and compute bill.
4. Display final total daily sales and remaining inventory.

4. Pseudocode

```
      FOR i = 1 to orders: IF valid_item AND quantity <= stock[idx]: stock[idx] -= quantity total_sa
les += prices[idx] * quantity ELSE: Print "Order rejected"
```

5. Flowchart

```
[Start] → [Display Menu] → [Input Orders] → <Stock Available?> —Yes→ [Deduct Stock & Add Bill]
↓ No
[Print Order Rejected] → [Display Total Sales & Remaining Stock] → [End]
```

6. Python Source Code (case09_canteen_stock.py)

```
      items = ["Sandwich", "Samosa", "Juice"] prices = [50, 15, 25] stock = [10, 20, 12] total_sales
= 0 print("Canteen Menu") for i in range(len(items)): print(i + 1, items[i], "- Rs.", prices[i], "|
Stock:", stock[i]) orders = int(input("Enter number of orders: ")) for i in range(orders): item_numb
er = int(input("enter item number(1-3):")) quantity = int(input("Enter quantity: ")) if item_number
< 1 or item_number > len(items) or quantity <= 0: print("Invalid order.") else: index = item_number
- 1 if quantity > stock[index]: print("Not enough stock. Order rejected.") else: bill = prices[inde
x] * quantity stock[index] -= quantity total_sales += bill print("Order accepted. Bill: Rs.", bill)
print("\nDaily Sales: Rs.", total_sales) print("Remaining Stock:") for i in range(len(items)): print
(items[i], ":", stock[i])
```

7. Test Cases & Sample Output

Order Input	Execution Output
Item 2 (Samosa), Qty 5	Order accepted. Bill: Rs. 75
Item 1 (Sandwich), Qty 15	Not enough stock. Order rejected.

8. Conclusion

Demonstrates list index mapping and interactive state updates for retail applications.

CASE STUDY 10: HOLLOW DIAMOND PATTERN GENERATOR

1. Problem Statement & Objectives

Generate a symmetrical hollow diamond pattern using star (`*`) characters based on input size n.

2. Assumptions & Scope

- Diamond size n bounded: $2 \leq n \leq 20$.
- Uses formula `2 * i - 3` for inside spaces calculation.

3. Algorithm

1. Input diamond size n; validate range [2, 20].
2. Top half: loop i from 1 to n; print leading spaces and stars.
3. Bottom half: loop i from n-1 down to 1; print corresponding inverted lines.

4. Pseudocode

```
FOR i = 1 to n: IF i == 1 THEN Print spaces + "*" ELSE Print spaces + "*" + inside_spaces + "*" REPEAT inverted loop for i = n-1 DOWNTO 1
```

5. Flowchart

```
[Start] → [Input Size n (2-20)] → [Loop Upper Half (1 to n)] → [Print Upper Lines]
↓
[Loop Lower Half (n-1 down to 1)] → [Print Inverted Lines] → [End]
```

6. Python Source Code (case10_numeric_pattern.py)

```
n = int(input("Enter diamond size (2-20): ")) if n < 2 or n > 20: print("Please enter a size between 2 and 20.") else: for i in range(1, n + 1): spaces = n - i if i == 1: print(" " * spaces + "*") else: inside_spaces = 2 * i - 3 print(" " * spaces + "*" + " " * inside_spaces + "*") for i in range(n - 1, 0, -1): spaces = n - i if i == 1: print(" " * spaces + "*") else: inside_spaces = 2 * i - 3 print(" " * spaces + "*" + " " * inside_spaces + "*")
```

7. Test Cases & Sample Output (n = 4)

```
* * * * * * * * * * * *
```

8. Conclusion

Demonstrates pattern formatting, nested loop ranges, and string repetition in Python.